

mf-kernels vs MFEM partial assembly and libCEED: a like-for-like benchmark

Author: Mohit Prajapati **Operators:** CEED BP1 (mass) and BP3 (Poisson), 3D hex, order $p = 1..8$, single core.

1. Purpose

mf-kernels is a small set of hand-tuned matrix-free tensor-product contraction kernels. This benchmark answers one concrete question: on the two standard CEED bake-off operators, how does the mf-kernels apply compare, on the same machine and with identical accounting, to MFEM native partial assembly and to libCEED's CPU backends? The aim is an honest measurement, not a headline.

2. Setup

Hardware / toolchain. AMD EPYC 7763 (Zen 3), single core used, AVX2 and FMA, no AVX-512. L1d 32 KiB, L2 512 KiB, L3 32 MiB. gcc 13.3.0, flags -O3 -march=native. Runs pinned with taskset -c 0, OMP_NUM_THREADS=1, OMP_PLACES=cores. The node is a cloud instance (1 physical core, 2 SMT threads); timing uses warmup plus 21 repeats with median and inter-quartile spread to absorb jitter.

Discretization. Order- p Gauss-Lobatto nodal H1 basis ($n_d = p+1$ nodes/dim). Gauss-Legendre quadrature, $n_q = p+2$ points/dim (rule order $2p+3$, the CEED BP convention), so the 1D operator is rectangular $(p+2) \times (p+1)$. Structured $n \times n \times n$ hex mesh on the unit cube, sized to roughly 0.5 M unique DOFs per p .

Operators. BP1 mass $v = G^T B^T (w \det J) B G u$. BP3 Poisson $v = G^T B^T D B G u$ with D the symmetric metric $w \det J J^{-1} J^{-T}$.

Implementations. (1) mf-kernels apply, variants naive / avx2 / avx2_blocked / even-odd; (2) MFEM native PA; (3) libCEED /cpu/self/avx/blocked; (4) libCEED /cpu/self/xsmm/blocked (LIBXSMM). MFEM built with libCEED + LIBXSMM, same compiler and flags.

Accounting. Throughput in unique global DOFs/sec. GFLOP/s (mf-kernels axis) uses one shared standard sum-factorization FLOP count across all variants, so the even-odd algorithmic saving shows as a wall-clock gain, not a smaller denominator.

3. Correctness

Two independent checks, both clean.

- **MFEM-independent** (validate_bp): the sum-factorized BP1/BP3 operator against a dense $O(p^6)$ reference built from a self-contained GLL/GL basis, all four variants, $p = 1..8$. Worst max-abs difference $8.9e-15$.
- **Against MFEM** (diff_bp): mf-kernels applied to MFEM's own input vector and element restriction, compared to MFEM's output vector, with a convention-independent invariant (BP1 mass of ones = unit-cube volume = 1.0; BP3 stiffness of ones = 0) confirming the discretization and scale match.

	BP1 worst	BP3 worst
max-abs over p , all variants	2.4e-17	3.8e-15
max-rel over p , all variants	2.5e-15	3.6e-15

All well under the $1e-13$ threshold. The four implementations compute the same operator with the same accounting; the throughput comparison below is like-for-like.

4. Results

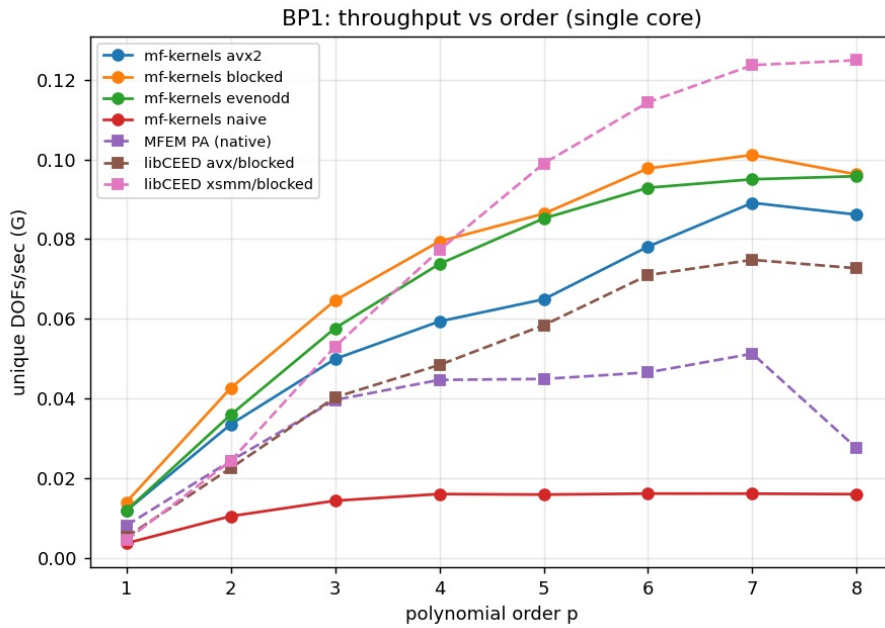
Single core, unique DOFs/sec in millions (M). mf-k = mf-kernels.

BP1 (mass).

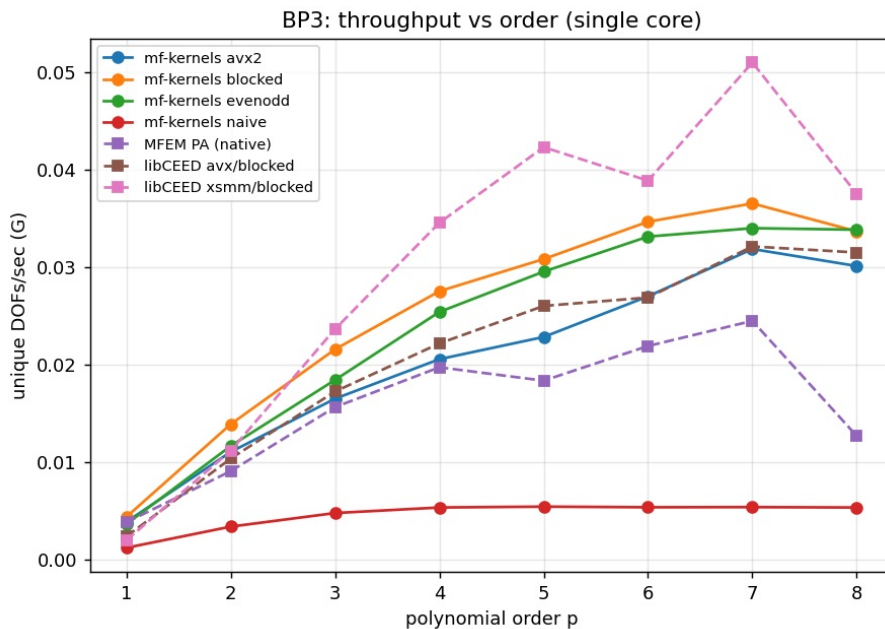
p	mf-k blocked	mf-k even-odd	mf-k avx2	MFEM PA	libCEED avx	libCEED xsmm
1	14.0	11.9	11.9	8.2	5.3	4.5
2	42.6	35.9	33.5	24.4	22.6	24.4
3	64.6	57.7	49.9	39.7	40.3	53.0
4	79.4	73.8	59.3	44.7	48.4	77.3
5	86.4	85.2	64.9	44.9	58.4	99.0
6	97.7	92.9	78.1	46.6	71.0	114.3
7	101.1	95.0	89.1	51.3	74.8	123.6
8	96.2	95.8	86.1	27.6*	72.7	124.9

BP3 (Poisson).

p	mf-k blocked	mf-k even-odd	mf-k avx2	MFEM PA	libCEED avx	libCEED xsmm
1	4.4	3.7	3.9	3.9	2.4	2.0
2	13.9	11.7	11.1	9.1	10.4	11.2
3	21.6	18.5	16.5	15.7	17.3	23.7
4	27.6	25.4	20.6	19.8	22.2	34.6
5	30.9	29.6	22.9	18.4	26.1	42.3
6	34.7	33.2	27.0	22.0	26.9	38.9
7	36.6	34.0	31.9	24.5	32.2	51.0
8	33.7	33.9	30.1	12.7*	31.5	37.6



BP1 throughput vs order



BP3 throughput vs order

* The MFEM native PA values at $p = 8$ (27.6 BP1, 12.7 BP3) are anomalously low relative to $p = 7$ and are most likely cloud-VM timing noise at that run; they are reported as measured and not interpreted further.

GFLOP/s (mf-kernels, shared standard count) rises from about 6 at $p = 1$ to about 17-18 at $p = 8$ for BP1 and about 19-20 for BP3, roughly 35-40 percent of the single-core AVX2 double-precision peak, consistent with a partly memory-bound full operator.

5. Interpretation (honest)

- **mf-kernels blocked is the strongest mf-kernels variant** and it beats MFEM native partial assembly at every order on both operators, often by close to 2x (BP1 $p = 5$: 86.4 vs 44.9 M DOFs/s; BP3 $p = 5$: 30.9 vs 18.4).
- **Against libCEED's own AVX blocked backend, mf-kernels blocked wins across the whole order range** on both operators (BP1 $p = 8$: 96.2 vs 72.7; BP3 $p = 8$: 33.7 vs 31.5). This is the fairest single comparison, since both batch elements into SIMD lanes.
- **The only implementation that beats mf-kernels is libCEED with LIBXSMM**, and only at higher order. mf-kernels leads at low order (BP1 $p \leq 3$, BP3 $p \leq 2$) and trails LIBXSMM by roughly 15-25 percent at high order (BP1 $p = 8$: 96 vs 125). Being within that margin of a production JIT GEMM library, while ahead of the hand-vectorized backend and the native path, is the honest takeaway.
- **Even-odd does not beat blocked once gather/scatter is included.** At the bare contraction level even-odd halves FMAs and wins; at the full-operator level the gather/scatter and the pointwise D apply dilute that saving, and blocked (better register reuse) is generally ahead. Even-odd only edges blocked at the largest BP3 case ($p = 8$: 33.9 vs 33.7). This matches the expectation that the full operator is partly memory-bound.
- No claim is made that mf-kernels beats MFEM or libCEED as a system. The measurement is one operator apply on a structured, constant-geometry mesh.

6. Threats to validity

See methodology_flags.md. Most load-bearing: the $q = p+2$ convention; the three different SIMD strategies (cross-element batch vs libCEED blocked vs MFEM native); unique-vs-E-vector DOF counting; the shared GFLOP/s convention; single core only (mf-kernels has no thread-safe scatter yet); the cloud-VM timing environment; and AVX2-only mf-kernels on a Zen 3 part that has no AVX-512 anyway, so no ISA gap here.

7. Reproduce

```
bash scripts/build.sh      # LIBXSMM + libCEED + MFEM + binaries
bash scripts/run.sh        # env capture, both sides, 1e-13 check, plots
```